King Saud University
College of Computer and Information Sciences

Department of Computer Science

CSC 212 Data Structures Project Report – 2nd Semester 2025-2026

Student Advising System

Phase 1

Authors

Ghina Alsubaie	445203058	Student [Constructor, setters & getters, getFirstName(), toString(), compareTo()] StudentList [add(), findByID(), findByName(),findByNameContains(), findByEmail(), findByMajor(), findByYearLevel(), getAll(), deleteById(), size()]

1. Introduction

The Student Advising System is a Java project designed to manage students and academic events such as meetings and workshops. The system supports adding, searching, and deleting students, scheduling events, and preventing scheduling conflicts. The project applies object-oriented programming concepts including inheritance, interfaces, and encapsulation, while using custom data structures such as linked lists. This report explains the system specifications, implementation logic, and time complexity analysis of the main classes and methods.

2. Specification

ADT LinkedList: Specification (Implemented 2 helper methods)

Elements: The elements are of generic type <T>. (The elements are placed in nodes for linked list implementation).

Structure: the elements are linearly arranged. The first element is called head, there is an element called current.

Domain: the number of elements in the list is bounded therefore the domain is finite. Type name of elements in the domain: LinkedList

Operations:

1- **Method `insertAtFront (T val)`**
requires: none.
input: val of type T.
results: if the list is empty, val is inserted as the first element and current is set to head. Otherwise val is inserted at the front of the list, head is updated to the new node, and current is set to the new head.
output: none.

2- **Method `insertBefore (T val)`**
requires: none.
input: val of type T.
results: if the list is empty or current is head, insertAtFront is called. Otherwise, the list is traversed from head to find the node before current, val is inserted before current, and current is set to the new node.
output: none.

ADT Student: Specification

Elements: The elements are the attributes of a student: studentId (int), name (String), email (String), major (String), yearLevel (int), notes (String), and schedule (LinkedList of IEvent).

Structure: a single Student object representing one student. The schedule field is a LinkedList that stores the student's events linearly.

Domain: the number of Student objects is bounded therefore the domain is finite. Type name of elements in the domain: Student

Operations:

1. **Method** `Student(int studentId, String name, String email, String major, int yearLevel, String notes)`
   requires: studentId is unique.
   input: studentId (int), name (String), email (String), major (String), yearLevel (int), notes (String).
   results: a new Student object is created with the given attributes and an empty schedule.
   output: none.

2. **Method** `getName(String name)`
   requires: none.
   input: none.
   results: retrieves the name of the student.
   output: name.

3. **Method** `setName(String name)`
   requires: none.
   input: name of type String.
   results: updates the student's name to the given value.
   output: none.

4. **Method** `getStudentId(int studentId)`
   requires: none.
   input: none.
   results: retrieves the unique identifier of the student.
   output: studentId.

5. **Method** `getEmail(String email)`
   requires: none.
   input: none.
   results: retrieves the email address of the student.
   output: email.

6. **Method** `setEmail(String email)`

requires: none.
input: email of type String.
results: updates the student's email to the given value.
output: none.

7. **Method `getMajor(String major)`**
requires: none.
input: none.
results: retrieves the major of the student.
output: major.

8. **Method `setMajor(String major)`**
requires: none.
input: major of type String.
results: updates the student's major to the given value.
output: none.

9. **Method `getYearLevel(int yearLevel)`**
requires: none.
input: none.
results: retrieves the year level of the student.
output: yearLevel.

10. **Method `setYearLevel(int yearLevel)`**
requires: yearLevel is a positive integer.
input: yearLevel of type int.
results: updates the student's year level to the given value.
output: none.

11. **Method `getNotes(String notes)`**
requires: none.
input: none.
results: retrieves the notes associated with the student.
output: notes.

12. **Method `setNotes(String notes)`**
requires: none.
input: notes of type String.
results: updates the student's notes to the given value.
output: none.

13. **Method `getFirstName(String firstName)`**
requires: none.

input: none.
results: if name is null or empty returns an empty string. If no space is found returns the full name. Otherwise returns the substring of name before the first space.
output: first name as a String.

14. **Method `getSchedule(LinkedList schedule)`**
    requires: none.
    input: none.
    results: retrieves the student's list of scheduled events.
    output: schedule.

15. **Method `compareTo(int)`**
    requires: other is not null.
    input: other of type IStudent.
    results: compares this student to another student by studentId. Returns 1 if this studentId is greater, -1 if less, 0 if equal.
    output: 1, -1, or 0.

16. **Method `toString(String)`**
    requires: none.
    input: none.
    results: constructs a formatted string containing all student information including name, studentId, email, major, yearLevel, and notes.
    output: formatted String.

ADT StudentList: Specification

Elements: The elements are of type IStudent. The elements are placed in nodes for linked list implementation.

Structure: the elements are linearly arranged in ascending order by studentId. The first element is called head, there is an element called current.

Domain: the number of elements in the list is bounded therefore the domain is finite. Type name of elements in the domain: StudentList

Operations:

1. **Method `add(IStudent student)`**
   requires: student is not null.
   input: student of type IStudent.

results: checks for duplicate studentId. If the list is empty inserts at front. Otherwise traverses the list to find the correct position maintaining ascending order by studentId and inserts accordingly.
output: true if insertion was successful, false if studentId already exists.

2. **Method `findById(int studentId)`**
   requires: none.
   input: studentId of type int.
   results: traverses the list searching for a student with the matching studentId. Stops as soon as the student is found. Checks the last element separately after the loop.
   output: the matching IStudent if found, null otherwise.

3. **Method `findByName(String fullName)`**
   requires: none.
   input: fullName of type String.
   results: traverses the entire list and collects all students whose name matches fullName exactly, case insensitive. Checks the last element separately after the loop.
   output: LinkedList of all matching IStudent objects, empty list if none found.

4. **Method `findByNameContains (String partialName)`**
   requires: none.
   input: partialName of type String.
   results: traverses the entire list and collects all students whose name contains partialName, case insensitive. Checks the last element separately after the loop.
   output: LinkedList of all matching IStudent objects, empty list if none found.

5. **Method `findByEmail (String email)`**
   requires: none.
   input: email of type String.
   results: traverses the list searching for a student with the matching email, case insensitive. Stops as soon as the student is found. Checks the last element separately after the loop.
   output: the matching IStudent if found, null otherwise.

6. **Method `findByMajor (String major)`**
   requires: none.
   input: major of type String.
   results: traverses the entire list and collects all students whose major matches the given major, case insensitive. Checks the last element separately after the loop.
   output: LinkedList of all matching IStudent objects, empty list if none found.

7. **Method `findByYearLevel (int yearLevel)`**
   requires: none.
   input: yearLevel of type int.
   results: traverses the entire list and collects all students whose year level matches the given value. Checks the last element separately after the loop.

output: LinkedList of all matching IStudent objects, empty list if none found.

8. **Method `getAll(LinkedList students)`**
   requires: none.
   input: none.
   results: retrieves the entire list of students.
   output: students LinkedList.

9. **Method `deleteById (int studentId)`**
   requires: none.
   input: studentId of type int.
   results: traverses the list searching for a student with the matching studentId. If found removes the student from the list. Stops as soon as the student is deleted. Checks the last element separately after the loop.
   output: true if student was found and deleted, false otherwise.

10. **Method `size (int count)`**
    requires: none.
    input: none.
    results: traverses the entire list counting each element. The last element is counted separately after the loop.
    output: total number of students in the list.

ADT DateTime: Specification

Elements: The elements are date and time values represented by year, month, day, hour, and minute of type int.

Structure: The elements are stored together as one DateTime object representing a specific date and time.

Domain: The domain is finite because the values are limited to valid ranges for dates and times.Type name of elements in the domain: DateTime

Operations:

1. **Method** DateTime(int year, int month, int day, int hour, int minute)
   requires: valid date and time values
   input: year, month, day, hour, minuts.
   results: validates the date and time values. If all values are valid, they are assigned to the object attributes.
   output: none.

2. **Method `format()`**
   requires: none.
   input: none.
   results: creates and returns a formatted date and time string.
   output: String value.

3. **Method `compareTo(DateTime other)`**
   requires: other must not be null.
   input: other of type IDateTime.
   results: compares the current DateTime object with another DateTime object using year, month, day, hour, and minute values.
   output: int value.

4. **Method `getYear()`**
5. **Method `getMonth()`**
6. **Method `getDay()`**
7. **Method `getHour()`**
8. **Method `getMinute()`**

ADT Event: Specification

Elements: The elements are event objects that store an event ID, title, start date/time, end date/time, and location.

Structure: Event is an abstract parent class. It cannot be instantiated directly. Its data is shared by child classes such as Meeting and Workshop.

Domain: The domain is finite because the number of events stored in the system is limited by the program memory. Type name of elements in the domain: Event.

Operations:

1- **Method** Event(int id, String title, IdateTime startDateTime, IdateTime endDateTime, String loction)
   requires: title is not null or empty, start and end date/time are not null, and start is before end.
   input: id, title, startDateTime, endDateTime, location.

results: creates an event object and initializes its main attributes. If location is null, it becomes an empty string.
output: none.

2- **Method `getEventId()`**
3- **Method `getTitle()`**
4- **Method `setTitle()`**
5- **Method `getStartDateTime()`**
6- **Method `getEndDateTime()`**
7- **Method `getLoction()`**
8- **Method `getEventType ()`**
9- **Method `toString ()`**
10- **Method `compareTo(IEvent other)`**
requires: other must not be null.
input: other of type Event.
results: compares events alphabetically by title, ignoring case.
output: integer comparison result.
11-**Method `conflictsWith(IEvent other)`**
requires: other must not be null.
input: other of type Event.
results: checks whether this event overlaps with the other event.
output: true if the time intervals overlap, false otherwise.

ADT EventList: Specification

Elements: The elements are objects of type IEvent.

Structure: The events are stored in a linked list and maintained in alphabetical order based on event title.

Domain: The number of events is finite because the list size is limited by the available memory. Type name of elements in the domain: EventList.

Operations:
1- **Method `EventList()`**
2- **Method** addEvent(Ievent event)
requires: event is not null.
input: event of type IEvent.
results: inserts the event into the list in alphabetical order using compareTo. If

the list is empty, the event is inserted directly.
output: true if insertion succeeds, false otherwise.

3- **Method** removeEventByld(int eventId)
requires: none.
input: eventId of type int.
results searches the list for an event with the given ID and removes it if found.
output: true if the event is removed, false otherwise

4- **Method** `getAllAlphabetically()`

5- **Method** findByTitle(String Title)
requires: none.
input: title of type String.
results searches for all events whose title matches the given title ignoring case sensitivity and stores them in a new linked list.
output: LinkedList<IEvent> containing matching events.

6- **Method** findByStudentName(String studentFullName)
requires: none.
input: studentFullName of type String.
results searches all meetings and workshops to find events involving a student with the given name. For meetings, the student is checked directly. For workshops, all participants are traversed and compared. Matching events are added to a result list.
output: LinkedList<IEvent> containing matching events.

7- **Method** Size()
requires: none.
input: none.
results traverses the linked list and counts the number of stored events.
output: integer representing the total number of events.

ADT Meeting: Specification

Elements: The elements are meeting objects. Each meeting contains the common event information inherited from Event, and one student of type IStudent.

Structure: Meeting is a child class of the abstract class Event. It represents a one-to-one event linked to a single student.

Domain: The number of meeting objects is finite because the number of events in the system is limited by memory. Type name of elements in the domain: Meeting.

Operations:

1. **Method** Meeting()
2. **Method** `getStuden()`
3. **Method** `setStudent(IStudent Student)`

4. **Method** `getEventType()`
5. **Method** `hasStudent()`

   requires: none.
   input: studentId of type int.
   Results: checks whether the meeting contains a student with the given ID
   output: true if the meeting has that student, false otherwise.

6. **Method** `toString()`

ADT Workshop: Specification

Elements: The elements are workshop objects. Each workshop contains common event information inherited from Event, and a list of student participants of type IStudent.

Structure: Workshop is a child class of the abstract class Event. The participants are stored linearly inside a custom LinkedList<IStudent>.

Domain: The number of workshop objects and participants is finite because the list size is limited by available memory. Type name of elements in the domain: Workshop.

Operations:

1. **Method** Workshop()

2. **Method** `getParticipants()`

3. **Method** `addParticipants(Istudent Student)`

   requires: student is not null.
   input student of type IStudent.
   results: if the participants list is empty, the student is inserted directly. Otherwise, the list is traversed to check for duplicate student ID. If the student is not already registered, the student is added to the participants list.

output: true if the student is added, false otherwise.

4. **Method** `removeParticipantsById(int StudentId)`
   requires: none.
   input studentId of type int.
   results: searches the participants list for a student with the given ID. If found, the student is removed from the list.
   output: true if the student is removed, false otherwise.

5. **Method** `isEmpty(int StudentId)`
   requires: none.
   input none.
   results: checks whether the participants list is empty.
   output: true if the workshop has no participants, false otherwise.

6. **Method** `getEventType()`
7. **Method** `hasStudent(int StudentId)`
8. **Method** `ToSring()`

ADT AdvisingSystem: Specification

Elements: The elements are student and event records. Students are stored in StudentList, and events are stored in EventList.

Structure: The system connects two main domains: the student domain and the event domain. It uses the student list to search and manage students, and the event list to schedule and print meetings/workshops.

Domain: The number of students and events is finite because the lists are limited by available memory. Type name of elements in the domain: AdvisingSystem.

Operations:

1- **Method** AdvisingSystem()
   requires: none.
   input: none.
   results: creates an empty student list and an empty event list.
   output: none.

2- **Method** loadStudentsFromCSV(String studentsFilePath)

requires: the file path exists and follows the expected CSV format.
input: studentsFilePath of type String.
results: reads student records from the CSV file, creates Student objects, and adds them to the system.
output: true if loading succeeds, false otherwise.

3- **Method** loadEventsFromCSV(String eventsFilePath)
requires: none.
input: eventsFilePath of type String.
results: currently returns false because event loading is not implemented.
output: false.

4- **Method** addStudent(IStudent student)
requires: student is not null.
input: student of type IStudent.
results: adds the student to the student list if possible.
output: true if added, false otherwise.

5- **Method** searchStudentById(int studentId)
requires: none.
input: studentId of type int.
results: searches the student list for a student with the given ID.
output: matching student if found, null otherwise.

6- **Method** searchStudentByEmail(String email)
requires: none.
input: email of type String.
results: searches the student list for a student with the given email.
output: matching student if found, null otherwise.

7- **Method** printStudentsByMajor(String major)
requires: none.
input: major of type String.
results: finds and prints all students with the given major.
output: printed student list.

8- **Method** printStudentsByYearLevel(int yearLevel)
requires: none.
input: yearLevel of type int.
results: finds and prints all students in the given year level.
output: printed student list.

9- **Method** printStudentsByName(String fullName)

requires: none.
input: fullName of type String.
results: finds and prints all students whose full name exactly matches the input.
output: printed student list.

10- **Method** printStudentsByPartialName(String partialName)
requires: none.
input: partialName of type String.
results: finds and prints students whose names contain the given partial name.
output: printed student list.

11- **Method** printAllStudents()
requires: none.
input: none.
results: prints all students stored in the system.
output: printed student list.

12- **Method** deleteStudent(int studentId)
requires: none.
input: studentId of type int.
results: searches for the student and deletes the student if found.
output: true if deleted, false otherwise.

13- **Method** scheduleMeeting(…)
requires: the student must exist and the new meeting must not conflict with the student's schedule.
input: title, startDateTime, endDateTime, location, and studentId.
results: creates a Meeting object, adds it to the event list, and inserts it into the student's schedule.
output: true if scheduled, false otherwise.

14- Method scheduleWorkshop(…)
requires: all students must exist and the workshop must not conflict with any listed student's schedule.
input: title, startDateTime, endDateTime, location, and array of student IDs.
results: creates a Workshop object, adds participants, inserts the workshop into each student's schedule, and adds the workshop to the event list.
output: true if scheduled, false otherwise.

15- **Method** hasConflict(IStudent student, IEvent newEvent)
requires: student and newEvent are not null.
input: student and newEvent.
results: checks the student's schedule to determine whether the new event overlaps with an existing event.
output: true if a conflict exists, false otherwise.

16- **Method** printEventDetailsByTitle(String title)
requires: none.
input: title of type String.
results: finds and prints all events with the given title.
output: printed event list.

17- **Method** printEventDetailsByStudentName(String studentName)
requires: none.
input: studentName of type String.
results: finds and prints all events involving a student with the given name.
output: printed event list.

18- **Method** printWorkshopStudents(String workshopTitle)
requires: none.
input: workshopTitle of type String.
results: finds workshops with the given title and prints their participants.
output: printed participants.

19- **Method** printAllEventsAlphabetically()
requires: none.
input: none.
results: prints all events stored alphabetically by title.
output: printed event list.

20- **Method** printStudentList(LinkedList list)
requires: list is not null.
input: linked list of students.
results: prints each student in the list, or prints a message if the list is empty.
output: printed students.

21- **Method** printEventList(LinkedList list)
requires: list is not null.
input: linked list of events.
results: prints each event in the list, or prints a message if the list is empty.
output: printed events

1. **ADT AdvisingSystemPhase1**: Specification

Elements: The elements are user menu choices and input values used to interact with the advising system.

Structure: The class contains the main method and uses a loop-based command-line menu. It connects the user interface with the AdvisingSystem class.

Domain: The number of menu operations is finite during one program execution. Type name of elements in the domain: AdvisingSystemPhase1.

Operations:

1- **Method** main(String[] args)
requires: none.
input: command-line arguments and user input from Scanner.
results: starts the advising system, loads student and event CSV files, displays the menu repeatedly, and calls the correct system methods based on the user's choice.
output: printed menu, prompts, success messages, search results, student details, and event details.

2- **Method** parseDateTime(String text)
requires: text must be in the format MM/DD/YYYY HH:MM.
input: text of type String.
results: splits the date and time values, converts them to integers, and creates a DateTime object.
output: IDateTime object.

3. Design

The Student Advising System is a large system that manages multiple domains including students and events.

The system is divided into two main categories: entity classes that represent real-world objects, and data structure classes that manage collections of those objects.

The **`Student`** class represents a single student in the system, storing all relevant information including ID, name, email, major, year level, notes, and schedule. The student ID cannot be changed after creation to ensure each student can always be uniquely identified.

The **`LinkedList`** class is the foundational data structure of the system. It stores elements one after another where each element knows the location of the next one. It keeps track of the first element and the element currently being worked on.

The **`StudentList`** class manages a collection of **`Student`** objects using **`LinkedList`** as its internal storage. It provides search operations including searching by ID, name, partial name, email, major, and year level.

Design Decisions:
Two key design decisions were made. First, students are always stored in order of their student ID, keeping the list organized and predictable. Second, two helper methods **`insertAtFront`** and **`insertBefore`** were added to **`LinkedList`** to support this ordered storage, this keeps list manipulation logic inside the list class where it belongs, and keeps **`StudentList`** focused on student

The event domain was designed using inheritance to reduce repeated code and improve maintainability. The Event class represents the shared behavior of all event types. It stores the common attributes including event ID, title, start date and time, end date and time, and location. It also provides shared methods such as comparing events alphabetically and checking scheduling conflicts between events.

The Meeting and Workshop classes extend the Event class. The Workshop class stores participants using a linked list of students, allowing dynamic insertion and removal of participants.

The EventList class manages all events in the system using a linked list as the internal storage structure. Events are inserted alphabetically by title using the compareTo method inherited from the Event class. The class also supports searching for events by title and by student name, as well as removing events using their unique ID.

A scheduling conflict mechanism was included to ensure that students cannot be assigned overlapping events. Before a meeting or workshop is scheduled, the system traverses the student's schedule and checks whether the new event overlaps with an existing one using the conflictsWith method. If a conflict exists, the event is rejected to maintain a valid schedule.

Several design decisions were made in the event domain. First, inheritance was used to group common event behavior inside a single abstract class rather than duplicating code in Meeting and Workshop. Second, workshops use a linked list to manage participants dynamically without requiring a fixed size. Third, event conflict checking was implemented inside the Event class so that all event types follow the same scheduling logic consistently across the system.

4. Implementation

Sorted Insertion in StudentList

The most critical implementation detail is maintaining sorted order by student ID when adding a new student. The **`add()`** method traverses the list until it finds the correct position, then decides whether to insert before or after the current element based on ID comparison:

```
while (!students.last() && student.getStudentId() >
students.retrieve().getStudentId()) {
    students.findNext();
}
if (student.getStudentId() < students.retrieve().getStudentId())
    students.insertBefore((Student) student);
else
    students.insert((Student) student);
```

This required two helper methods to be added to **`LinkedList`**: **`insertAtFront`** for empty list insertion and **`insertBefore`** for inserting behind the current position.

insertBefore Helper Method

Since the linked list only has a forward pointer and cannot go backwards, **`insertBefore`** had to traverse from head to find the node just before current in order to rewire the pointers correctly.

Search Strategy

Methods that return a single result (**`findById`**, **`findByEmail`**, **`deleteById`**) use a **`found`** and delete flag in the while condition to exit as soon as a match is found. Methods that return multiple results (`findByName`, `findByMajor`, `findByYearLevel`, `findByNameContains`) visit every element since

Event Scheduling and Conflict Detection

One of the most important implementation details in the event domain is the scheduling conflict mechanism. Before a meeting or workshop is added, the system checks the student's existing schedule to ensure that no overlapping event exists. This validation is performed by traversing the student's scheduled events and comparing the start and end times of each event with the new event. If a conflict is detected, the scheduling operation immediately stops and the event is rejected.

Alphabetical Event Storage

The EventList class maintains events in alphabetical order based on event titles. During insertion, the list is traversed until the correct alphabetical position is found using the compareTo method implemented in the Event class. This ensures that events remain sorted automatically without requiring an additional sorting algorithm later.

Workshop Participant Management

The Workshop class manages participants using a linked list of students. Before adding a participant, the system traverses the list to ensure that the student is not already registered in the workshop. This prevents duplicate participant entries and keeps workshop data consistent. Participant removal is implemented by searching for the matching student ID and deleting the corresponding node from the linked list.

Inheritance Implementation

Inheritance was implemented by creating an abstract Event class that stores all common event properties and behaviors. The Meeting and Workshop classes extend this parent class and implement their own specialized functionality. This reduced repeated code and centralized shared event logic such as event comparison, formatting, and conflict checking.

CSV File Loading

Student information is loaded from CSV files using file reading classes. Each line from the file is split into attributes such as student ID, name, email, major, and notes. These values are then converted into Student objects and inserted into the system automatically. Exception handling was implemented to safely handle invalid files or formatting errors during loading.

Traversal-Based Processing

Most operations in the project rely on linked list traversal because the custom linked list structure does not support direct indexing. Searching, insertion, deletion, conflict checking, and event filtering are all implemented by moving node by node through the list until the required condition is satisfied. This approach follows the behavior of sequential linked list structures taught in the course.

5. Performance analysis

We gave a more detailed Performance analysis using **Student** and **StudentList** Classes to showcase how to calculate the growth rate function:

Student Class

Student()	S/E	Freq	Total
public Student(int studentId, String name, String email, String major, int yearLevel, String notes) {	1	1	1
this.studentId = studentId;	1	1	1
this.name = name;	1	1	1
this.email = email;	1	1	1
this.major = major;	1	1	1
this.yearLevel = yearLevel;	1	1	1
this.notes = notes;	1	1	1
this.schedule = new LinkedList<IEvent>(); }	1	1	1
Total	8		
O	O(1)		
getName()	S/E	Freq	Total
public String getName() {	1	1	1
return name; }	1	1	1
Total	2		
O	O(1)		
setName()	S/E	Freq	Total
public void setName(String name) {	1	1	1
this.name = name; }	1	1	1
Total	2		
O	O(1)		

getStudentId()	S/E	Freq	Total
public int getStudentId() {	1	1	1
return studentId; }	1	1	1
Total	2		
O	O(1)		

getEmail()	S/E	Freq	Total
public String getEmail() {	1	1	1
return email; }	1	1	1
Total	2		
O	O(1)		

setEmail()	S/E	Freq	Total
public void setEmail(String email) {	1	1	1
this.email = email; }	1	1	1
Total	2		
O	O(1)		

getMajor()	S/E	Freq	Total
public String getMajor() {	1	1	1
return major; }	1	1	1
Total	2		
O	O(1)		

setMajor()	S/E	Freq	Total
public void setMajor(String major) {	1	1	1
this.major = major; }	1	1	1
Total	2		
O	O(1)		

getYearLevel()	S/E	Freq	Total
public int getYearLevel() {	1	1	1
return yearLevel; }	1	1	1
Total	2		
O	O(1)		

setYearLevel()	S/E	Freq	Total
public void setYearLevel(int yearLevel) {	1	1	1
this.yearLevel = yearLevel; }	1	1	1
Total	2		
O	O(1)		

getNotes()	S/E	Freq	Total
public String getNotes() {	1	1	1
return notes; }	1	1	1
Total	2		
O	O(1)		

setNotes()	S/E	Freq	Total
public void setNotes(String notes) {	1	1	1
this.notes = notes; }	1	1	1
Total	2		
O	O(1)		

getFirstName()	S/E	Freq	Total
public String getFirstName() {	1	1	1
if (name == null || name.isEmpty())	1	1	1
return "";	1	1	1
int space = name.indexOf(" ");	1	1	1
if (space == -1)	1	1	1
return name;	1	1	1
else	0	-	0
return name.substring(0, space); }	1	1	1
Total	7		
O	O(1)		

toString()	S/E	Freq	Total
public String toString() {	1	1	1
return "\nStudent found!\n" +	1	1	1
"Name: " + this.name + "\n" +	1	1	1
"Student ID: " + this.getStudentId() + "\n" +	1	1	1
"Email Address: " + this.getEmail() + "\n" +	1	1	1
"Major: " + this.getMajor() + "\n" +	1	1	1
"Year Level: " + this.yearLevel + "\n" +	1	1	1
"Notes: " + this.notes; }	1	1	1
Total	8		
O	O(1)		

compareTo()	S/E	Freq	Total
public int compareTo(IStudent other) {	1	1	1
if (this.studentId > other.getStudentId()) return 1;	1	1	1
if (this.studentId < other.getStudentId()) return -1;	1	1	1
return 0; }	1	1	1
Total	4		
O	O(1)		

StudentList Class

add()	S/E	Freq	Total
public boolean add(IStudent student) {	1	1	1
if (findById(student.getStudentId()) != null) {	1	1	1
return false; }	1	1	1
if (students.empty()) {	1	1	1
students.insertAtFront((Student) student);	1	1	1
return true; }	1	1	1
students.findFirst();	1	1	1
while(!students.last()&& student.getStudentId()>students.retrieve().getStudentId()) {	1	n	n
students.findNext();}	1	n-1	n-1
If(student.getStudentId()< students.retrieve().getStudentId()) {	1	1	1
students.insertBefore((Student) student); }	1	1	1
else {	0	-	0
students.insert((Student) student); }	1	1	1

return true; }	1	1	1
Total	2n + 9 + n(findById)		
O	O(n)		
findById()	**S/E**	**Freq**	**Total**
public IStudent findById(int studentId) {	1	1	1
IStudent student = null;	1	1	1
boolean found = false;	1	1	1
if (! students.empty()){	1	1	1
students.findFirst();	1	1	1
while (!students.last() && !found) {	1	n	n
if (studentId == students.retrieve().getStudentId()){	1	n-1	n-1
found = true;	0	-	0
student = students.retrieve();	0	-	0
} else	0	-	0
students.findNext();}	1	n-1	n-1
if (!found && (studentId == students.retrieve().getStudentId())){	1	1	1
found = true;	1	1	1
student = students.retrieve();	1	1	1
} }	0	-	0
return student;	1	1	1
}	0	-	0
Total	3n + 7		
O	O(n)		
findByName()	**S/E**	**Freq**	**Total**
public LinkedList<IStudent> findByName(String fullName) {	1	1	1
LinkedList<IStudent> data = new LinkedList<IStudent>();	1	1	1
if (!students.empty()) {	1	1	1
students.findFirst();	1	1	1
while (!students.last()) {	1	n	n
if (students.retrieve().getName().equalsIgnoreCase(fullName)) {	1	n-1	n-1
data.insert(students.retrieve());	1	n-1	n-1
}	0	-	0
students.findNext();}	1	n-1	n-1
if (students.retrieve().getName().equalsIgnoreCase(fullName)) {	1	1	1
data.insert(students.retrieve());	1	1	1
}}	0	-	0
return data;	1	1	1
}	0	-	0
Total	4n + 3		
O	O(n)		
findByNameContains()	**S/E**	**Freq**	**Total**
public LinkedList<IStudent> findByNameContains(String partialName) {	1	1	1
LinkedList<IStudent> data = new LinkedList<IStudent>();	1	1	1
if (!students.empty()) {	1	1	1

students.findFirst();	1	1	1
while (!students.last()) {	1	n	n
if (students.retrieve().getName().toLowerCase().contains(partialName.toLowerCase())) {	1	n-1	n-1
data.insert(students.retrieve());	1	n-1	n-1
}	0	-	0
students.findNext();}	1	n-1	n-1
If(students.retrieve().getName().toLowerCase().contains(partialName.toLowerCase())) {	1	1	1
data.insert(students.retrieve());	1	1	1
} }	0	-	0
return data;	1	1	1
}	0	-	0
Total	4n + 3		
O	O(n)		
findByEmail()	**S/E**	**Freq**	**Total**
public IStudent findByEmail(String email) {	1	1	1
IStudent student = null;	1	1	1
boolean found = false;	1	1	1
if (!students.empty()) {	1	1	1
students.findFirst();	1	1	1
while (!students.last() && !found) {	1	n	n
if (email.equalsIgnoreCase(students.retrieve().getEmail())) {	1	n-1	n-1
found = true;	0	-	0
student = students.retrieve();	0	-	0
} else {	0	-	0
students.findNext();	1	n-1	n-1
} }	0	-	0
if (!found && email.equalsIgnoreCase(students.retrieve().getEmail())) {	1	1	1
found = true;	1	1	1
student = students.retrieve();	1	1	1
}}	0	-	0
return student;	1	1	1
}	0	-	0
Total	3n + 7		
O			
findByMajor()	**S/E**	**Freq**	**Total**
public LinkedList<IStudent> findByMajor(String major) {	1	1	1
LinkedList<IStudent> data = new LinkedList<IStudent>();	1	1	1
if (!students.empty()) {	1	1	1
students.findFirst();	1	1	1
while (!students.last()) {	1	n	n
if (students.retrieve().getMajor().equalsIgnoreCase(major)) {	1	n-1	n-1
data.insert(students.retrieve());	1	n-1	n-1
}	0	-	0
students.findNext();}	1	n-1	n-1
if (students.retrieve().getMajor().equalsIgnoreCase(major)) {	1	1	1

data.insert(students.retrieve());	1	1	1
}}	0	-	0
return data;	1	1	1
}	0	-	0
Total	4n - 3		
O	O(n)		
findByYearLevel()	**S/E**	**Freq**	**Total**
public LinkedList<IStudent> findByYearLevel(int yearLevel) {	1	1	1
LinkedList<IStudent> data = new LinkedList<IStudent>();	1	1	1
if (!students.empty()) {	1	1	1
students.findFirst();	1	1	1
while (!students.last()) {	1	n	n
if (students.retrieve().getYearLevel() == yearLevel) {	1	n-1	n-1
data.insert(students.retrieve());	1	n-1	n-1
}	0	-	0
students.findNext();}	1	n-1	n-1
if (students.retrieve().getYearLevel() == yearLevel) {	1	1	1
data.insert(students.retrieve());	1	1	1
}}	0	-	0
return data;	1	1	1
}	0	-	0
Total	4n + 3		
O	O(n)		
getAll()	**S/E**	**Freq**	**Total**
public LinkedList<IStudent> getAll() {	1	1	1
return students;	1	1	1
}	0	-	0
Total	2		
O	O(1)		
deleteById()	**S/E**	**Freq**	**Total**
public boolean deleteById(int studentId)	1	1	1
{	0	-	0
boolean delete = false;	1	1	1
if (!students.empty()){	1	1	1
students.findFirst();	1	1	1
while (!students.last() && !delete)	1	n	n
{	0	-	0
if (studentId == students.retrieve().getStudentId()){	1	n-1	n-1
delete = true;	0	-	0
students.remove();	0	-	0
} else	0	-	0
students.findNext();	1	n-1	n-1
}	0	-	0
if (!delete && studentId == students.retrieve().getStudentId()){	1	1	1
delete = true;	1	1	1

students.remove();	1	1	1
} }	0	-	0
return delete;	1	1	1
}	0	-	0
Total	3n + 6		
O	O(n)		

size()	S/E	Freq	Total
public int size() {	1	1	1
int count = 0;	1	1	1
if (!students.empty()) {	1	1	1
students.findFirst();	1	1	1
while (!students.last()) {	1	n	n
count++;	1	n-1	n-1
students.findNext();	1	n-1	n-1
}	0	-	0
count++; }	1	1	1
return count;	1	1	1
}	0	-	0
Total	3n + 4		
O			

LinkedList Class

public LinkedList ()	O(1)	it performs a fixed number of instructions
public boolean empty ()	O(1)	it performs a fixed number of instructions
public boolean last ()	O(1)	it performs a fixed number of instructions
public boolean full ()	O(1)	it performs a fixed number of instructions
public void findFirst ()	O(1)	it performs a fixed number of instructions
public void findNext ()	O(1)	it performs a fixed number of instructions
public T retrieve ()	O(1)	it performs a fixed number of instructions
public void update (T val)	O(1)	it performs a fixed number of instructions
public void insert (T val)	O(1)	it performs a fixed number of instructions
public void insertAtFront(T val)	O(1)	it performs a fixed number of instructions
public void insertBefore(T val)	O(n)	loops from head to find the node before current
public void remove ()	O(n)	from head to find the node before current in order to rewire pointers

Event Class

public Event(...)	O(1)	it performs a fixed number of instructions
getEventId()	O(1)	it performs a fixed number of instructions
getTitle()	O(1)	it performs a fixed number of instructions
setTitle()	O(1)	it performs a fixed number of instructions

getStartDateTime()	O(1)	it performs a fixed number of instructions
getEndDateTime()	O(1)	it performs a fixed number of instructions
getLocation()	O(1)	it performs a fixed number of instructions
setLocation()	O(1)	it performs a fixed number of instructions
comperTo(IEvent other)	O(1)	it performs a fixed number of instructions
conflictsWith(IEvent other)	O(1)	it performs a fixed number of instructions
getEvetType()	O(1)	it performs a fixed number of instructions
toString()	O(1)	it performs a fixed number of instructions

EventList Class

public EventList()	O(1)	it performs a fixed number of instructions
addEvent(IEvent event)	O(n)	Loops through events to insert in order
removeEventById(int eventId)	O(n)	Loops until matching ID is found
getAllAlphabetically()	O(1)	it performs a fixed number of instructions
findByTitle(String title)	O(n)	Searches all event titles
findByStudentName(String studentFullName)	O(1)	Searches events and workshop participants
size()	O(n)	Counts all events in the list

Meeting Class

Meeting(...)	O(1)	it performs a fixed number of instructions
getStudent()	O(1)	it performs a fixed number of instructions
setStudent()	O(1)	it performs a fixed number of instructions
getEventType()	O(1)	it performs a fixed number of instructions
hasStudent()	O(1)	it performs a fixed number of instructions
toString()	O(1)	it performs a fixed number of instructions

WorkShop Class

Workshop(...)	O(1)	it performs a fixed number of instructions
getParticipants()	O(1)	it performs a fixed number of instructions
addParticipant()	O(n)	Loops through participants to check duplicates
removeParticipantById()	O(n)	Searches for matching participant ID
isEmpty()	O(1)	it performs a fixed number of instructions
getEventType()	O(1)	it performs a fixed number of instructions
hasStudent()	O(n)	Searches through participants
toString()	O(n)	Traverses all participants to build the string

DataTime Class

DateTime(...)	O(1)	it performs a fixed number of instructions

getYear()	O(1)	it performs a fixed number of instructions
getMonth()	O(1)	it performs a fixed number of instructions
getDay()	O(1)	it performs a fixed number of instructions
getHour()	O(1)	it performs a fixed number of instructions
getMinute()	O(1)	it performs a fixed number of instructions
format()	O(1)	it performs a fixed number of instructions
compareTo()	O(1)	it performs a fixed number of instructions

AdvisingSystem Class

AdvisingSystem()	O(1)	it performs a fixed number of instructions
loadStudentsFromCSV()	O(n)	Reads all students from the file once
loadEventsFromCSV()	O(1)	it performs a fixed number of instructions
addStudent()	O(n)	Depends on student list insertion/search
searchStudentById()	O(n)	Searches through students
searchStudentByEmail()	o O(n)	Searches through students
printStudentsByMajor()	o O(n)	Traverses matching students
printStudentsByYearLevel()	O(n)	Traverses matching students
printStudentsByName()	O(n)	Searches through students
printStudentsByPartialName()	O(n)	Searches through students
printAllStudents()	O(n)	Traverses all students
deleteStudent()	O(n)	Searches and deletes student
scheduleMeeting()	O(n)	Checks conflicts in student schedule
scheduleWorkshop()	$O(n^2)$	Loops through students and conflict checks
hasConflict()	O(n)	Traverses student schedule
printEventDetailsByTitle()	O(n)	Searches through events
printEventDetailsByStudentName()	$O(n \times m)$	Searches events and participants
printWorkshopStudents()	$O(n \times m)$	Traverses workshops and participants
printAllEventsAlphabetically()	O(n)	Traverses all events
printStudentList()	O(n)	Traverses student list
printEventList()	O(n)	Traverses event list

AdvisingSystemPhase1 Class

main()	$O(n^2)$	Calls searching, scheduling, and printing methods
parseDateTime()	O(1)	it performs a fixed number of instructions

6. Conclusion

In conclusion, the Student Advising System successfully applied object-oriented programming concepts and custom data structures to manage students, meetings, and workshops. The project implemented important operations such as searching, scheduling, deletion, and conflict checking using custom linked lists instead of Java built-in collections.

The project helped strengthen understanding of inheritance, interfaces, abstraction, linked lists, and algorithm analysis. One of the main challenges was handling traversal and sorted insertion manually, but this provided deeper practical experience with data structures.

In the future, the system could be improved by adding a graphical interface, database integration, and more advanced scheduling features. Overall, the project provided valuable hands-on experience in building a complete and organized software system.